



FitStudent מדריך שלב 2

חיבור האפליקציה ל-AI אמיתי · Perplexity · Google Gemini · Make.com · קבוצה 42

למילוי עצמאי ע"י הצוות

המשך לאחר שלב 1 (Alpha)

מדריך Step-by-Step

תוכן עניינים

- | | |
|---|---|
| 1. המטרה של שלב 2 | 7. Scenario E תוכנית אימונים (Gemini) |
| 2. הארכיטקטורה החדשה | 8. Scenarios C/D/F דוחות Telegram (אופציונלי) |
| 3. חשבונות, מפתחות והרשאות (Checklist) | 9. חיבור הצד-לקוח (Frontend Wiring) |
| 4. יסודות Make.com | 10. אבטחה, עלויות ומגבלות |
| 5. Scenario A ניתוח ארוחה (+ Gemini Vision) | 11. בדיקות ו-Rollout |
| (Perplexity) | 12. חלוקת עבודה לצוות |
| 6. Scenario B מהמקרר לצלחת | 13. נספח תבניות Prompt מלאות |

1 המטרה של שלב 2

בשלב 1 בנינו אפליקציה מלאה ועובדת, אבל יכולות ה-AI הן עדיין **מדומות (mock)**. שלב 2 מחליף אותן ב-AI אמיתי.

בשלב 1 (ראה תיעוד שלב 1) הפונקציות בקובץ `src/services/ai.js` מחזירות נתוני-דוגמה. בכל מקום כזה כתוב `// PHASE 2`. המטרה עכשיו:

- **צילום ארוחה** 🍎 → ניתוח אמיתי עם **Gemini Vision**, ואימות ערכים תזונתיים עם **Perplexity**.
- **מהמקרר לצלחת** 🧊 → זיהוי מרכיבים מתמונה (Gemini Vision) + יצירת מתכון מותאם (Gemini) + תמונת מנה (Imagen, אופציונלי).
- **תוכנית אימונים** 🏆 → יצירה דינמית ומותאמת-פרופיל עם **Gemini** (במקום התבניות הסטטיות).
- **(אופציונלי) דוחות יומיים/שבועיים ב-Telegram**. 📧

ה"דבק" שמחבר את הכל הוא **Make.com** פלטפורמת אוטומציה שבה נבנה "תרחישים" (Scenarios) שמקבלים בקשה מהאפליקציה, פונים ל-Gemini/Perplexity, ומחזירים תשובה.

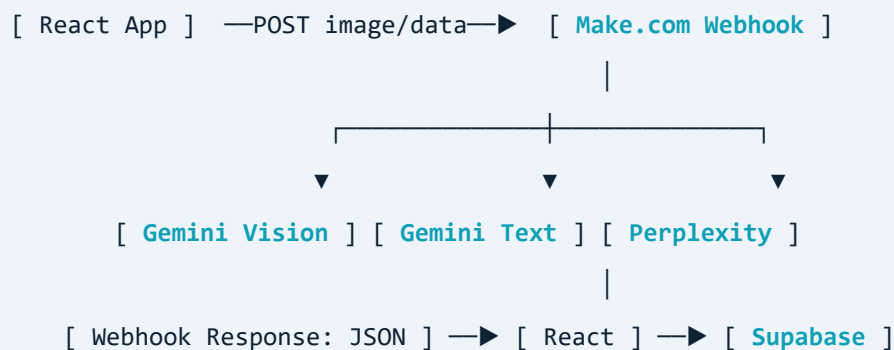
למה **Make.com** באמצע ולא קריאה ישירה ל-Gemini מהאפליקציה?

כי מפתחות ה-API (Gemini, Perplexity) הם **סודיים**. אם נשים אותם בקוד ה-Frontend כל מי שייכנס לאתר יוכל לגנוב אותם. **Make.com** שומר את המפתחות אצלו (בצד-שרת), והאפליקציה מדברת רק עם כתובת ה-Webhook הציבורית. כך המפתחות לעולם לא חשופים.

2 הארכיטקטורה החדשה

שינוי מינימלי: ה-Frontend ו-Supabase נשארים כמו שהם. מתווספת רק שכבת עיבוד-AI (Make.com).

עיקרון מנחה: **Make.com** הוא **שירות עיבוד בלבד** מקבל בקשה, מחזיר JSON. **האפליקציה היא זו שכותבת ל-Supabase** (בדיוק כמו היום), כך שאיננו נוגעים במנגנון השמירה והאבטחה (RLS) שכבר עובד.



חריג: התרחישים המתוזמנים (Telegram, פרק 8) רצים ברקע ללא האפליקציה לכן הם כן צריכים לקרוא נתונים מ-Supabase ישירות (עם מפתח service-role סודי).

3 חשבונות, מפתחות והרשאות

זה הפרק הכי חשוב להתחלה. צרו את כל החשבונות והמפתחות לפני שנוגעים בקוד. **סודי** = לעולם לא בקוד ה-Frontend ולא ב-GitHub.

מה	איפה משיגים	הרשאות / הערות	סוג	עלות
Make.com חשבון	make.com → Sign up	הירשמו עם מייל הקבוצה. בחרו Region (EU/US) שמרו עקביות.	חשבון	Free 1,000 פעולות/חודש)
Gemini API Key Google AI Studio	aistudio.google.com/app/apikey → Create API key	נדרש חשבון Google. המפתח מפעיל את Generative Language API. עובד ל-Vision ול-Text. ל-Imagen (תמונות) ייתכן ויידרש פרויקט עם חיוב (Billing).	סודי	Free tier נדיב; Imagen בתשלום
Perplexity API Key	perplexity.ai/settings/api → Generate	צריך להוסיף אמצעי תשלום / קרדיט כדי שהמפתח יעבוד. מודלים: sonar , sonar-pro .	סודי	בתשלום (זול; ~\$ לפי שימוש)
Supabase service_role רק לתרחישים מתוזמנים	Supabase → Project Settings → API → service_role	עוקף RLS וניגש לכל הנתונים לכן אך ורק בתוך Make.com , לעולם לא ב- Frontend. נחוץ רק	סודי מאוד	קיים

ל-Telegram (פרק 8).

Free	סודי	מחזיר token לשליחת הודעות. צריך גם chat_id של כל משתמש.	בטלגרם: דברו עם @BotFather → /newbot	Telegram Bot Token אופציונלי
—	ציבורי	אלה כתובות ציבוריות (לא סוד), נכניס אותן ל-Frontend כמשתני-סביבה.	נוצרים אוטומטית כשמוסיפים מודול Webhook בכל תרחיש	Webhook URLs נוצרים ב-Make

כלל הזהב לאבטחה: מפתחות מסומנים **סודי** נכנסים רק לתוך מודולים ב-Make.com. הם לעולם לא נכתבים בקוד, לא ב-.env שנדחף ל-GitHub, ולא נשלחים בצ'אט. ה-Frontend מכיר רק את כתובות ה-Webhook.

טיפ ארגוני: פתחו מסמך/כספת סיסמאות משותף לצוות (למשל פתק מאובטח) ושמרו בו את כל המפתחות במקום אחד, עם שם ברור לכל אחד.

4 יסודות Make.com

לפני בניית התרחישים המושגים שצריך להכיר.

מושא	מה זה
Scenario (תרחיש)	"צינור" של שלבים. כל פיצ'ר AS שלנו = תרחיש אחד.
Module (מודול)	שלב בודד בתרחיש למשל "קבל Webhook", "שלח בקשת HTTP", "החזר תשובה".
Webhook	מודול-טריגר שמייצר כתובת URL. כשהאפליקציה שולחת אליה POST התרחיש רץ.

המודול הגנרי שבו נקרא ל-Gemini ול-Perplexity (עם POST) (JSON).	HTTP > Make a request
המודול האחרון מחזיר JSON חזרה לאפליקציה.	Webhook Response
טריגר מבוסס-זמן (Cron) לדוחות Telegram המתוזמנים.	Scheduler
כפתור בתחתית Make להרצת התרחיש פעם אחת לצורך בדיקה/האזנה ל-Webhook.	Run once

זרימת עבודה כללית לכל תרחיש: מוסיפים Webhook → לוחצים Run once → שולחים בקשת-בדיקה מהאפליקציה (או מ-Postman) כדי ש-Make "ילמד" את מבנה הנתונים → מוסיפים את מודולי ה-HTTP ל-AI → ממפים את השדות → מוסיפים Webhook Response → מפעילים (Activate) את התרחיש.

5 Scenario A ניתוח ארוחה

קלט: תמונת ארוחה (base64). פלט: שם, פריטים, וקלוריות/מאקרו מאומתים. מחליף את

analyzeFoodImage

בניית התרחיש ב-Make

1 תרחיש חדש → מודול Add → Custom webhook → Webhook → תנו שם fitstudent-food . העתיקו את ה-URL.

2 Run once, ואז שלחו בקשת-בדיקה אל ה-URL עם גוף לדוגמה {"image_base64": "...", "meal_type": "lunch"} כדי ש-Make ילמד את המבנה.

3 מודול HTTP → Make a request אל Gemini Vision:

```
# URL (POST)
https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent

# Headers
Content-Type: application/json

# Body (JSON) מה Webhook-התמונה ב-inline_data.data
```

```
{
  "contents": [{
    "parts": [
      { "text": "[הוראת המערכת לזיהוי מזון ראה נספח, פרק 13]", },
      { "inline_data": { "mime_type": "image/jpeg", "data": "{{1.image_base64}}" } }
    ]
  }],
  "generationConfig": { "response_mime_type": "application/json" }
}
```

מודול **HTTP** אל **Perplexity** כדי לאמת את הערכים מול USDA:

4

```
# URL (POST) https://api.perplexity.ai/chat/completions
# Headers Authorization: Bearer {{PERPLEXITY_KEY}} · Content-Type: application/json
# Body
{
  "model": "sonar",
  "messages": [
    { "role": "system", "content": "Return ONLY JSON with corrected totals." },
    { "role": "user", "content": "USDA exact macros for: {{geminiItems}}. Return {calo"
  ]
}
```

מודול **JSON → Parse JSON** על תשובת Gemini ועל תשובת Perplexity (כי הן חוזרות כטקסט).

5

מודול **Webhook Response: Status 200**, גוף JSON שמאחד את התוצאות, וכותרת **Access-Control-Allow-Origin: *** (ראה אזהרת CORS למטה).

6

```
{
  "name": "{{gemini.name}}",
  "emoji": "🥗",
  "items": {{gemini.items}},
  "calories": {{perplexity.calories}},
  "protein_g": {{perplexity.protein_g}},
  "carbs_g": {{perplexity.carbs_g}},
  "fat_g": {{perplexity.fat_g}},
  "confidence": {{gemini.confidence}}
}
```

Activate את התרחיש. שמרו את ה-Webhook URL לפרק 9.

7

חשוב מבנה התשובה: ה-JSON שחוזר חייב להתאים בדיוק למה שהאפליקציה מצפה לו

2-`analyzeFoodImage`: `{name, emoji, items:[{name, weight_g}], calories, protein_g, carbs_g, fat_g, confidence}`. כך לא צריך לשנות את ה-UI בכלל.

CORS: הדפדפן חוסם בקשות חוצות-מקור ללא הכותרת הנכונה. ודאו שמודול ה-Webhook Response מחזיר `Access-Control-Allow-Origin: *`. אם הדפדפן שולח בקשת OPTIONS מקדימה (preflight) הוסיפו ב-Make ראוטר שמזהה `method = OPTIONS` ומחזיר 200 עם אותן כותרות CORS.

6 Scenario B מהמקרר לצלחת

שני שלבים: (1) זיהוי מרכיבים מתמונה, (2) יצירת מתכון מהמרכיבים. מחליף את `detectFridgeIngredients` ל-`suggestRecipesFromIngredients`.

B1 זיהוי מרכיבים (תמונה → רשימה)

1 תרחיש עם **Webhook** בשם `fitstudent-fridge-detect` שמקבל `{ image_base64 }`.

2 מודול **Gemini Vision** → **HTTP** (כמו ב-Scenario A) עם הוראה: "זזה את כל המרכיבים הנראים במקרר. החזר JSON: מערך מחרוזות בעברית".

3 **Webhook Response** → **Parse JSON** שמחזיר מערך פשוט: ביצים, "עגבנייה", "גבינה" `["לבנה"]`.

B2 יצירת מתכונים (רשימה → מתכונים)

1 תרחיש עם **Webhook** בשם `fitstudent-fridge-recipe` שמקבל `{ ingredients: [...], protein_target_g, remaining_protein_g }`.

2 מודול **Gemini Text** → **HTTP** (מודל `gemini-2.0-flash`) עם prompt ליצירת 2-3 מתכונים מותאמים (ראה נספח). החזר מערך מתכונים בפורמט שהאפליקציה מצפה לו.

3 **(אופציונלי, מתקדם)** מודול **Imagen** → **HTTP** ליצירת תמונת מנה לכל מתכון (דורש Billing). אם מדלגים משאירים את אימוג'י המנה הקיים.

Webhook Response שמחזיר מערך בפורמט של: `rankRecipesByIngredients`

4

```
[{recipe:{name,emoji,prep_time_min,servings,ingredients:[],steps:[],macros:
...}, have:[], missing:[], needsExtra:bool}]
```

למה שני תרחישים נפרדים? כי באפליקציה זה גם שני שלבים נפרדים: קודם רואים את המרכיבים שזוהו ועורכים אותם (תיקון טעויות זיהוי), ורק אז בונים מתכון. זה משאיר למשתמש שליטה בדיוק כפי שבנינו בשלב 1.

גודל תמונה: תמונות base64 כבדות. לפני השליחה כדאי לדחוס/להקטין את התמונה בצד-לקוח (למשל לרוחב מקס' 1024px) כדי לחסוך זמן ותעבורה. ראה הערה בפרק 9.

7 Scenario E תוכנית אימונים

קלט: פרופיל המשתמש. פלט: תוכנית שבועית מותאמת. מחליף את התבניות הסטטיות

ב- `generateWorkoutPlan`

1 תרחיש עם **Webhook** בשם `fitstudent-plan` שמקבל את הפרופיל:

```
{ "goal":"bulk", "experience":"intermediate", "workouts_per_week":4,
  "workout_type":"gym", "body_type":"meso", "weight_kg":80 }
```

2 מודול **HTTP → Gemini Text** (`gemini-2.0-flash`, `response_mime_type: application/json`) עם ה-prompt מהנספח, שמבקש להחזיר מערך של 7 ימים.

3 **Parse JSON → Webhook Response** שמחזיר מערך בפורמט המדויק שהאפליקציה כותבת ל- `workout_plan`:

```
[ { "day_of_week":"sunday", "workout_name":"חזה + כתפיים",
  "muscle_groups":"חזה, כתפיים", "workout_type":"gym",
  "workout_variant":"intermediate",
  "exercises":[ { "name":"להיצת חזה", "sets":4, "reps":8, "weight_kg":50, "description":"...
  { "day_of_week":"monday", "workout_name":"מנוחה", "exercises":[] }, ... ]
```

שימו לב לשדות החדשים משלב 1: התוכנית חייבת לכלול (gym/yoga/pilates/crossfit) `workout_type` -I `workout_variant` (רמה/סגנון), כי הוספנו אותם בשלב 1.3. בקשו מ-Gemini ב-prompt להחזיר אותם, ולכבד את `workout_type` שהמשתמש בחר.

Telegram Scenarios C/D/F דוחות 8

אופציונלי אך מרשים. תרחישים מתוזמנים שרצים ברקע ושולחים הודעות. כאן Make קורא מ-Supabase ישירות.

תרחיש	מתי	מה עושה
C דוח בוקר	כל יום 07:00	קורא יעדים + האימון של היום → Gemini כותב הצעת בוקר → שולח בטלגרם.
D התראת ערב	כל יום 20:00	מחשב כמה חלבון חסר היום → אם חסר, Perplexity ממליץ על נשנוש → טלגרם.
F דוח שבועי	ראשון 09:00	אגרגציה של 7 ימים → Gemini מנתח חוזקות ושיפורים → טלגרם.

איך Make קורא מ-Supabase

```
# HTTP → Make a request (GET) אל Supabase REST
URL: https://YOUR-PROJECT-REF.supabase.co/rest/v1/users?select=*
Headers: apikey: {{SUPABASE_SERVICE_ROLE}}
         Authorization: Bearer {{SUPABASE_SERVICE_ROLE}}
```

הכנה בצד-האפליקציה: במסך ההגדרות כבר קיים שדה Telegram. צריך להשלים: שמירת `telegram_id` של המשתמש ל-`users` (הפונקציה `registerTelegram` ב-`ai.js` תפנה ל-Webhook שמאמת את ה-`chat_id`). מומלץ להנחות את המשתמש לשלוח `/start` לבוט כדי לקבל את ה-`chat_id`.

מגבלת Free: תרחישים מתוזמנים צורכים פעולות מהמכסה החודשית של Make. בחבילה החינמית התדירות המינימלית היא 15 דקות, ויש תקרת פעולות מספיק לדמו, לא לקנה-מידה גדול.

9 חיבור הצד-לקוח (Frontend)

השינוי בקוד מינימלי וממוקד רק בקובץ `src/services/ai.js` ובמשתני-הסביבה. השאר לא משתנה.

א. הוספת כתובות ה-Webhook כמשתני-סביבה

ב- `.env` מקומי וגם ב-Vercel (Project → Settings → Environment Variables):

```
VITE_MAKE_WEBHOOK_FOOD=https://hook.eu2.make.com/xxxxx
VITE_MAKE_WEBHOOK_FRIDGE_DETECT=https://hook.eu2.make.com/yyyyy
VITE_MAKE_WEBHOOK_FRIDGE_RECIPE=https://hook.eu2.make.com/zzzzz
VITE_MAKE_WEBHOOK_PLAN=https://hook.eu2.make.com/wwwww
```

שימו לב: כתובות Webhook הן **ציבוריות** (לא סודיות) מותר שהן יהיו ב-Frontend. הסוד היחיד (מפתחות Gemini/Perplexity) חבוי בתוך Make. עדכנו גם את `.env.example` ואת `README.md`.

ב. עוטף קריאה גנרי + Fallback ל-mock

מוסיפים פונקציית עזר, ומחליפים את גוף ה-stubs. שומרים את ה-mock כ-**גיבוי** כדי שהאפליקציה לעולם לא תישבר:

```
// src/services/ai.js
const WEBHOOKS = {
  food: import.meta.env.VITE_MAKE_WEBHOOK_FOOD,
  fridgeDetect: import.meta.env.VITE_MAKE_WEBHOOK_FRIDGE_DETECT,
  fridgeRecipe: import.meta.env.VITE_MAKE_WEBHOOK_FRIDGE_RECIPE,
  plan: import.meta.env.VITE_MAKE_WEBHOOK_PLAN,
}

async function callWebhook(url, payload) {
  const res = await fetch(url, {
    method: 'POST',
```

```

    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  })
  if (!res.ok) throw new Error('webhook ' + res.status)
  return res.json()
}

export async function analyzeFoodImage(imageBase64) {
  try {
    if (!WEBHOOKS.food) throw new Error('no webhook')
    return await callWebhook(WEBHOOKS.food, { image_base64: imageBase64 })
  } catch (e) {
    console.warn('AI food failed, fallback to mock:', e)
    await sleep(800)
    return foodResults[Math.floor(Math.random() * foodResults.length)]
  }
}

```

למה Fallback? אם Make נופל / נגמרה המכסה / אין אינטרנט האפליקציה ממשיכה לעבוד עם נתוני-דמה במקום להישבר. זו "ירידה מבוקרת" (graceful degradation) חשיבה מערכתית שמרצים אוהבים.

ג. המרת התמונה ל-base64 (היכן שמעלים קובץ)

כיום הפונקציה `onFile` מעבירה `null`. צריך לקרוא את הקובץ ל-base64 לפני השליחה:

```

function fileToBase64(file) {
  return new Promise((resolve, reject) => {
    const r = new FileReader()
    r.onload = () => resolve(String(r.result).split(',')[1]) // data: בלי תחילית
    r.onerror = reject
    r.readAsDataURL(file)
  })
}

```

ד. מפת השינויים מה מחליפים בכל פונקציה

פונקציה ב- ai.js	Webhook	שולח	מקבל
<code>analyzeFoodImage</code>	FOOD	<code>{ image_base64 }</code>	אובייקט ארוחה
<code>detectFridgeIngredients</code>	FRIDGE_DETECT	<code>{ image_base64 }</code>	מערך מרכיבים

מערך מתכונים
מדורגים

{ ingredients,
protein_target_g }

FRIDGE_RECIPE

suggestRecipesFromIngredients

מערך 7 ימים

{ goal, experience, ... }

PLAN

← generateWorkoutPlan

שינוי קטן ב-Onboarding: כיום `generateWorkoutPlan` היא סינכרונית. כשתהפוך ל-`async` (קריאת Webhook), צריך להוסיף `await` בקריאה לה ב-`Onboarding.jsx` וב-`Workouts.jsx` (החלפת תוכנית). מסכי הטעינה כבר קיימים רק לוודא שהם מוצגים בזמן ההמתנה.

10 אבטחה, עלויות ומגבלות

- **מפתחות סודיים רק ב-Make:** Gemini, Perplexity, ו-`service_role` של Supabase לעולם לא ב-Frontend ולא ב-GitHub.
- **CORS:** כל Webhook Response חייב `Access-Control-Allow-Origin: *` (ולטפל ב-OPTIONS preflight אם צריך).
- **הגנה על Webhook:** הכתובות ציבוריות. כדי למנוע שימוש לרעה אפשר להוסיף "מפתח משותף" פשוט (header סודי קטן) שה-Frontend שולח ו-Make בודק, או להגביל קצב.
- **עלויות:** Gemini מכסה חינמית נדיבה. Perplexity בתשלום לפי שימוש (זול). Imagen בתשלום, לכן אופציונלי. עקבו אחרי המכסה ב-Make (1,000 פעולות/חודש בחינם).
- **גודל תמונות:** דחסו/הקטינו לפני שליחה ($\geq 1024px$) חוסך זמן, תעבורה ומכסה.
- **פרטיות:** תמונות מזון נשלחות ל-Gemini ונתוני תזונה ל-Perplexity. מסך הפרטיות בשלב 1 כבר מציין זאת ודאו שהוא מעודכן.
- **אל תשנו את RLS:** מנגנון האבטחה של Supabase משלב 1 נשאר Make לא נוגע בו (חוץ מהתרחישים המתוזמנים עם `service_role`).

11 בדיקות ו-Rollout

- 1. **בדיקת Webhook לבד:** שלחו POST מ-Postman/דפדפן לכל Webhook ובדקו שה-JSON שחוזר תקין ובפורמט הנכון.
- 2. **בדיקה מקומית:** מלאו `.env`, הריצו `npm run dev`, ובדקו כל פיצ'ר (צילום ארוחה, מקרר, תוכנית).
- 3. **בדיקת Fallback:** כבו זמנית את אחד התרחישים ודאו שהאפליקציה נופלת חזרה ל-mock בלי לקרוס.
- 4. **פריסה:** הוסיפו את משתני-הסביבה ב-`git push` → Vercel → פריסה אוטומטית.
- 5. **בדיקה בענן:** חזרו על הבדיקות באתר החי, כולל ממכשיר נייד אמיתי (מצלמה).
- 6. **ניטור:** ב-History → Make רואים כל הרצה, מה נשלח ומה חזר כלי ניפוי מצוין.

תחום	משימות	דורש
חשבונות ומפתחות	פתיחת Make, Gemini, Perplexity; ריכוז המפתחות בכספת משותפת.	—
תרחיש A (ארוחה)	בניית התרחיש + prompt + בדיקה.	Make, Gemini, Perplexity
תרחיש B (מקרר)	שני התרחישים (זיהוי + מתכון), Imagen אופציונלי.	Make, Gemini
תרחיש E (אימונים)	בניית תרחיש התוכנית + ולידציית פורמט.	Make, Gemini
Frontend	עדכון <code>ai.js</code> , base64, async, env, Fallback.	הקוד
Telegram (אופציונלי)	בוט + 3 תרחישים מתוזמנים + קריאת Supabase.	BotFather, Make, service_role

סדר מומלץ: התחילו מתרחיש A (הפשוט והמרשים ביותר), הוכיחו שהזרימה עובדת מקצה-לקצה (אפליקציה→Gemini→Make→חזרה), ורק אז שכפלו את הדפוס לשאר התרחישים.

העתקה-הדבקה לתוך מודולי Gemini/Perplexity ב-Make. בקשו תמיד JSON תקין בלבד.

Gemini Vision זיהוי ארוחה (Scenario A)

```
אתה תזונאי מדייק. זהה כל פריט מזון בתמונה. החזר JSON תקין בלבד,
בלי markdown ובלי הסברים, במבנה הבא:
}
"name": "שם קצר של הארוחה בעברית",
"items": [{"name": "שם הפריט", "weight_g": מספר}],
"calories": מספר, "protein_g": מספר, "carbs_g": מספר, "fat_g": מספר,
"confidence": "0-100"
{
```

Perplexity אימות ערכים (Scenario A)

Return ONLY JSON. Using USDA FoodData Central, give the exact combined macros for these items and weights: {{items}}.

Schema: { "calories": n, "protein_g": n, "carbs_g": n, "fat_g": n }

Gemini Vision זיהוי מרכיבי מקרר (Scenario B1)

זהה את כל מרכיבי המזון הנראים בתמונת המקרר. החזר JSON תקין בלבד: מערך של מחרוזות בעברית, למשל ["ביצים", "עגבנייה", "גבינה לבנה"]. אל תכלול פריטים שאינם מזון.

Gemini Text יצירת מתכונים (Scenario B2)

```
אתה שף-תזונאי לסטודנטים בתקציב נמוך. בהינתן המרכיבים: {{ingredients}},
ויעד חלבון יומי {{protein_target_g}} גרם, צור 3 מתכונים מהירים.
החזר JSON תקין בלבד מערך של אובייקטים:
}]
"recipe": { "name": "...", "emoji": "🍳", "prep_time_min": n, "servings": n
, ["..."] : "ingredients": ["..."], "steps"
, { macros: { "calories": n, "protein_g": n, "carbs_g": n, "fat_g": n } }
, "have": ["מרכיבים שכבר יש למשתמש"]
, "missing": ["מרכיבים שצריך להשלים עד 2"]
needsExtra: true/false
```

[[
מיון מתכונים עם missing ריק קודם.

Gemini Text תוכנית אימונים (Scenario E)

בנה תוכנית אימונים שבועית מותאמת. נתונים:
מטרה={{goal}}, ניסיון={{experience}}, ימים בשבוע={{workouts_per_week}},
סוג אימון={{workout_type}} (gym/yoga/pilates/crossfit).
החזר JSON תקין בלבד מערך של 7 ימים. כל יום:
{
 "day_of_week": "sunday..saturday",
 "workout_name": "...",
 "muscle_groups": "...",
 "workout_type": "{{workout_type}}",
 "workout_variant": "רמה/סגנון",
 "exercises": [{
 "name": "...",
 "sets": n,
 "reps": n,
 "weight_kg": n,
 "description": "..."
 }]
} = workout_name = "מנוחה" = exercises = []
פזר את ימי האימון באופן מאוזן על פני השבוע.

טיפ: אם Gemini מחזיר לפעמים טקסט סביב ה-JSON, השתמשו ב-`response_mime_type: "application/json"` ב-`generationConfig`, או הוסיפו ב-Make מודול קטן שמחלץ את ה-JSON (regex) בין { } ל- { } לפני ה-`Parse`.